



Introduction to Data Structures

Definition

- **Data:** Collection of **raw facts**.
- **Data structure** is representation of the logical relationship existing between individual elements of data.
- **Data structure** is a specialized format for organizing and storing data in memory that considers not only the elements stored but also their relationship to each other.

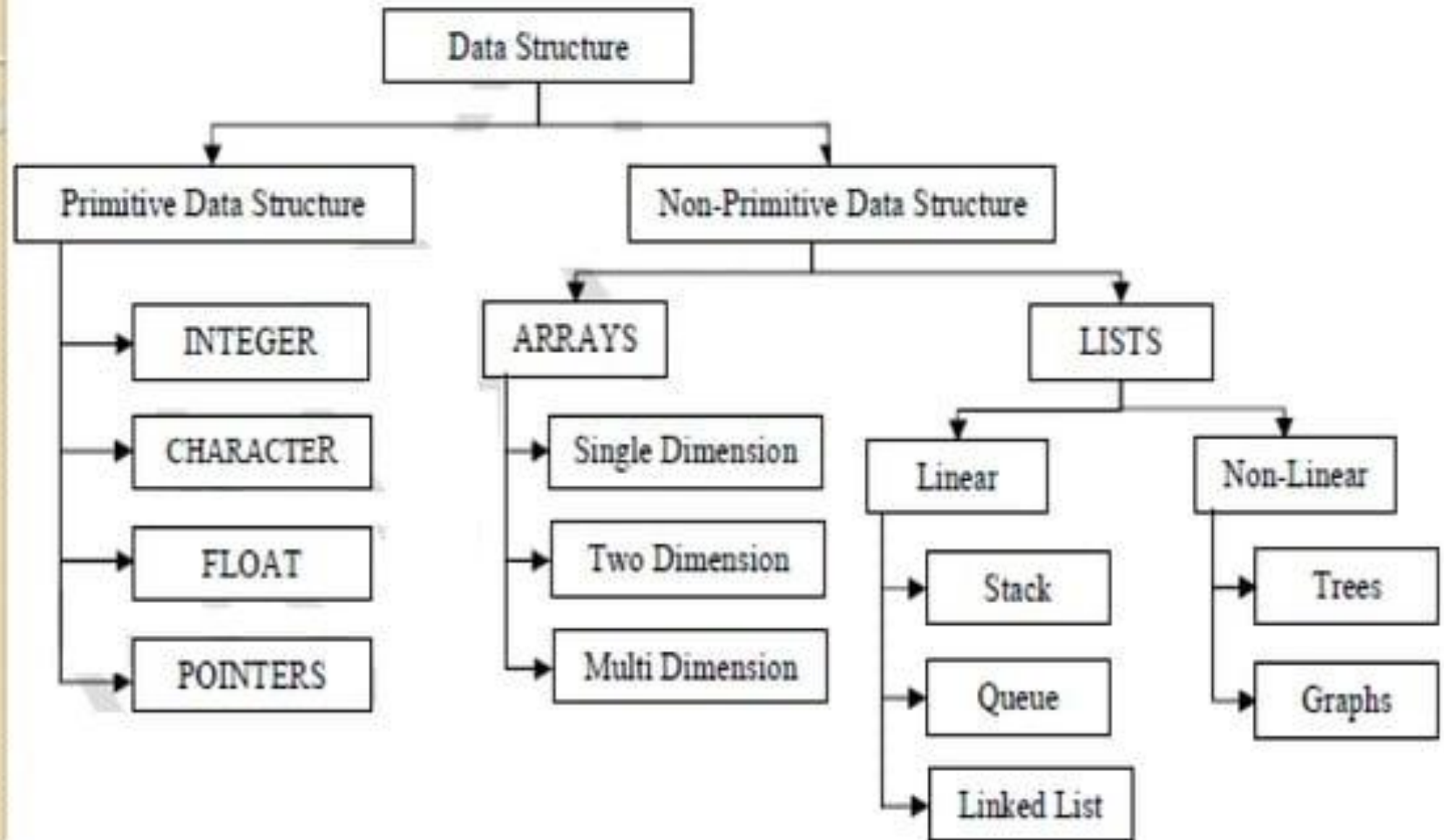
Introduction

- Data structure affects the design of both structural & functional aspects of a program.
Program=algorithm + Data Structure
- You know that a algorithm is a step by step procedure to solve a particular function.

Classification of Data Structure

- Data structure are normally divided into two broad categories:
 - Primitive Data Structure
 - Non-Primitive Data Structure

Classification of Data Structure



Primitive Data Structure

- There are basic structures and directly operated upon by the machine instructions.
- *Data structures that are directly operated upon the machine-level instructions are known as primitive data structures.*
- Integer, Floating-point number, Character constants, string constants, pointers etc, fall in this category.

Primitive Data Structure

- The most commonly used operation on data structure are broadly categorized into following types:
 - Create
 - Selection
 - Updating
 - Destroy or Delete

Non-Primitive Data Structure

- There are more sophisticated data structures.
- *The Data structures that are derived from the primitive data structures are called Non-primitive data structure.*
- The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (different type) data items.

Non-Primitive Data Structure

Linear Data structures:

- *Linear Data structures are kind of data structure that has homogeneous elements.*
- The data structure in which elements are in a sequence and form a liner series.
- Linear data structures are very easy to implement, since the memory of the computer is also organized in a linear fashion.
- Some commonly used linear data structures are **Stack, Queue and Linked Lists.**

Non-Linear Data structures:

- *A Non-Linear Data structures is a data structure in which data item is connected to several other data items.*
- Non-Linear data structure may exhibit either a hierarchical relationship or parent child relationship.
- The data elements are not arranged in a sequential structure.
- The different non-linear data structures are **trees and graphs.**

Non-Primitive Data Structure

- The most commonly used operation on data structure are broadly categorized into following types:
 - Traversal
 - Insertion
 - Selection
 - Searching
 - Sorting
 - Merging
 - Destroy or Delete

Different between them

- **A primitive data structure** is generally a basic structure that is usually built into the language, such as an integer, a float.
- **A non-primitive data structure** is built out of primitive data structures linked together in meaningful ways, such as a or a linked-list, binary search tree, AVL Tree, graph etc.

Description of various Data Structures : Arrays

- An array is defined as a set of finite number of homogeneous elements or same data items.
- It means an array can contain one type of data only, either all integer, all float-point number or all character.

One dimensional array:

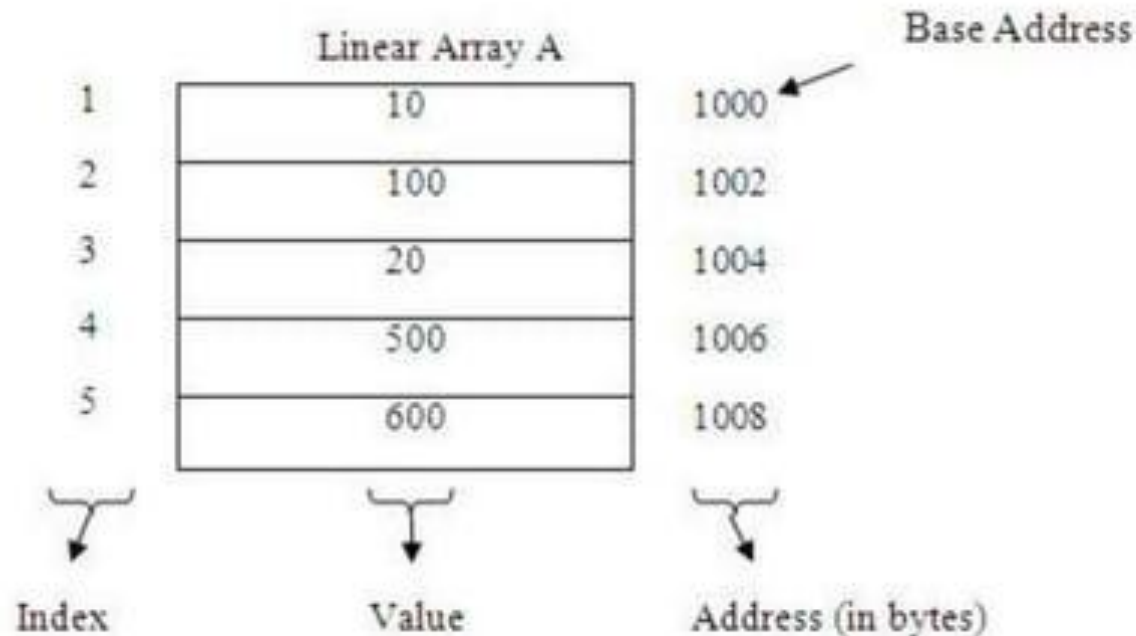
- *An array with only one row or column is called one-dimensional array.*
- It is finite collection of n number of elements of same type such that:
 - can be referred by indexing.
 - The syntax Elements are stored in continuous locations.
 - Elements x to define one-dimensional array is:
- **Syntax: Datatype Array_Name [Size];**
- Where,
Datatype : Type of value it can store (Example: int, char, float)
Array_Name: To identify the array.
- Size : The maximum number of elements that the array can hold.

Arrays

- Simply, declaration of array is as follows:
`int arr[10]`
- Where int specifies the data type or type of elements arrays stores.
- “arr” is the name of array & the number specified inside the square brackets is the number of elements an array can store, this is also called sized or length of array.

Represent a Linear Array in memory

- The elements of linear array are stored in consecutive memory locations. It is shown below:



Arrays

- The elements of array will always be stored in the consecutive (continues) memory location.
- The number of elements that can be stored in an array, that is the size of array or its length is given by the following equation:
$$(\text{Upperbound}-\text{lowerbound})+1$$
- For the above array it would be $(9-0)+1=10$, where 0 is the lower bound of array and 9 is the upper bound of array.
- Array can always be read or written through loop.

```
For(i=0;i<=9;i++)  
{  scanf("%d",&arr[i]);  
    printf("%d",arr[i]); }
```

Arrays types

- Single Dimension Array
 - Array with one subscript
- Two Dimension Array
 - Array with two subscripts (Rows and Column)
- Multi Dimension Array
 - Array with Multiple subscripts

Basic operations of Arrays

- Some common operation performed on array are:
 - Traversing
 - Searching
 - Insertion
 - Deletion
 - Sorting
 - Merging

Traversing Arrays

- **Traversing:** It is used to access each data item exactly once so that it can be processed.

E.g.

We have linear array A as below:

- 1 2 3 4 5
- 10 20 30 40 50

Here we will start from beginning and will go till last element and during this process we will access value of each element exactly once as below:

A [1] = 10
A [2] = 20
A [3] = 30
A [4] = 40
A [5] = 50

ALGORITHM: Traversal (A, LB, UB) A is an array with Lower Bound LB and Upper Bound UB.

Step 1: for LOC = LB to UB do
Step 2: PROCESS A [LOC]
 [End of for loop]
Step 3: Exit

Insertion into Array

- Insertion:** It is used to add a new data item in the given collection of data items.

E.g. We have linear array A as below:

1	2	3	4	5
10	20	50	30	15

New element to be inserted is 100 and location for insertion is 3. So shift the elements from 5th location to 3rd location downwards by 1 place. And then insert 100 at 3rd location. It is shown below:

1	2	3	4	5	6
10	20	50	30	15	15

1	2	3	4	5	6
10	20	50	30	30	15

1	2	3	4	5	6
10	20	50	50	30	15

1	2	3	4	5	6
10	20	100	50	30	15

ALGORITHM: Insert (A, N, ITEM, Pos) A is an array with N elements. ITEM is the element to be inserted in the position Pos.

Step 1: for $I = N-1$ down to Pos

$A[I+1] = A[I]$

[End of for loop]

Step 2: $A[Pos] = ITEM$

Step 3: $N = N+1$

Step 4: Exit

Deletion from Array

- **Deletion:** It is used to delete an existing data item from the given collection of data items.

For example: Let A[4] be an array with items 10, 20, 30, 40, 50 stored at consecutive locations. Suppose item 30 has to be deleted at position 2. The following procedure is applied.

- Copy 30 to ITEM, i.e. Item = 30.
- Move Number 40 to the position 2.
- Move Number 50 to the position 3.

A[0]	10
A[1]	20
A[2]	30
A[3]	40
A[4]	50

A[0]	10
A[1]	20
A[2]	
A[3]	40
A[4]	50

A[0]	10
A[1]	20
A[2]	40
A[3]	
A[4]	50

A[0]	10
A[1]	20
A[2]	40
A[3]	50
A[4]	

ALGORITHM: Delete (A, N, ITEM, Pos) A is an array with N elements. ITEM is the element to be deleted in the position Pos and it is stored into variable Item.

- Step 1: ITEM = A [Pos]
Step 2: for I = Pos down to N-1
 A[I] = A[I+1]
 [End of for loop]
Step 3: N = N-1
Step 4: Exit


```
// Insert an element in an array
```

```
include <iostream>
```

```
using namespace std;
```

```
const int MAX_SIZE = 100;
```

```
int main()
```

```
{ int arr[MAX_SIZE];
```

```
    int n = 0; // Current size of the array
```

```
    // Insertion
```

```
        cout << "Enter the number of elements to insert: ";
```

```
        cin >> n;
```

```
        cout << "Enter " << n << " elements:\n";
```

```
        for (int i = 0; i < n; ++i)
```

```
        {
```

```
            cin >> arr[i];
```

```
        }
```

```
    // Traversal
```

```
        cout << "Array elements are:\n";
```

```
        for (int i = 0; i < n; ++i)
```

```
        {
```

```
            cout << arr[i] << " ";
```

```
        }
```

```
        cout << endl;
```

// Deletion

```
int pos;
cout << "Enter the position to delete an element: ";
cin >> pos;
if (pos < 0 || pos >= n)
{
    cout << "Invalid position!\n";
}
else
{
    for (int i = pos; i < n - 1; ++i)
        { arr[i] = arr[i + 1]; }
    --n;
    // Decrease the size of the array
    cout << "Element at position " << pos << " is deleted.\n";
}
// Traversal after deletion
cout << "Array elements after deletion are:\n";
for (int i = 0; i < n; ++i)
{
    cout << arr[i] << " ";
}
cout << endl;
return 0;
```

Merging from Array

- **Merging:** It is used to combine the data items of two sorted files into single file in the sorted form

We have sorted linear array A as below:

1	2	3	4	5	6
10	40	50	80	95	100

And sorted linear array B as below:

1	2	3	4
20	35	45	90

After merging merged array C is as below:

1	2	3	4	5	6	7	8	9	10
10	20	35	40	45	50	80	90	95	100

Two dimensional array

- *A two dimensional array is a collection of elements and each element is identified by a pair of subscripts. (A[3] [3])*
- The elements are stored in continuous memory locations.
- The elements of two-dimensional array as rows and columns.
- The number of rows and columns in a matrix is called as the order of the matrix and denoted as mxn.
- The number of elements can be obtained by multiplying number of rows and number of columns.

	A[0]	A[1]	A[2]
A[0]	10	20	30
A[1]	40	50	60
A[2]	70	80	90

Representation of Two Dimensional Array:

- A is the array of order $m \times n$. To store $m \times n$ number of elements, we need $m \times n$ memory locations.
- The elements should be in contiguous memory locations.
- There are two methods:
 - Row-major method
 - Column-major method

Two Dimensional Array:

- Row-Major Method: All the first-row elements are stored in sequential memory locations and then all the second-row elements are stored and so on. Ex: A[Row][Col]
- Column-Major Method: All the first column elements are stored in sequential memory locations and then all the second-column elements are stored and so on. Ex: A [Col][Row]

1000	10	A[0][0]
1002	20	A[0][1]
1004	30	A[0][2]
1006	40	A[1][0]
1008	50	A[1][1]
1010	60	A[1][2]
1012	70	A[2][0]
1014	80	A[2][1]
1016	90	A[2][2]

Row-Major Method

1000	10	A[0][0]
1002	40	A[1][0]
1004	70	A[2][0]
1006	20	A[0][1]
1008	50	A[1][1]
1010	80	A[2][1]
1012	30	A[0][2]
1014	60	A[1][2]
1016	90	A[2][2]

Col-Major Method

Advantages of Array:

- It is used to represent multiple data items of same type by using single name.
- It can be used to implement other data structures like linked lists, stacks, queues, tree, graphs etc.
- Two-dimensional arrays are used to represent matrices.
- Many databases include one-dimensional arrays whose elements are records.

Disadvantages of Array

- We must know in advance the how many elements are to be stored in array.
- Array is static structure. It means that array is of fixed size. The memory which is allocated to array cannot be increased or decreased.
- Array is fixed size; if we allocate more memory than requirement then the memory space will be wasted.
- The elements of array are stored in consecutive memory locations. So insertion and deletion are very difficult and time consuming.

```
#include <stdio.h>
int main()
{
    int b[2][3];
    int i,j;
    printf("Enter elements into 2-D array: ");
    for(i=0;i<2;i++)
    {
        for(j=0;j<3;j++)
        {
            scanf("%d" , &b[i][j]);
        }
    }
    return 0;
}
```


Insert an element at position in 2D array

```
#include <stdio.h>
int main()
{ int b[2][3];
  int i,j,num;
  printf("Enter elements into 2-D array: ");
  for(i=0;i<2;i++)
  { for(j=0;j<3;j++)
    { scanf("%d" , &b[i][j]); }
  }
  printf("Enter the value of row and coulmn number :");
  scanf("%d %d", &i,&j);
  printf("Enter the number you want to update with: ");
  scanf("%d" , &num);
  b[i][j]=num;
  for(i=0;i<2;i++)
  { for(j=0;j<3;j++)
    { printf("\t %d" , b[i][j]);
    } printf("\n"); }
  return 0;
}
```

Deletion in 2D array

```
#include <stdio.h>
int main()
{ int b[2][3],i,j,num,x;
printf("Enter elements into 2-D array: ");
for(i=0;i<2;i++)
{ for(j=0;j<3;j++)
{ scanf("%d" , &b[i][j]); } }
printf("Enter the value of row number :");
scanf("%d", &x);
for(i=0;i<2;i++)
{ if(i==x) { for(j=0;j<3;j++)
{ if((i+1)<2) {
printf("\t%d" , b[i+1][j]);
} } i++;}
else
{ for(j=0;j<3;j++)
{ printf("\t%d" , b[i][j]); }
} printf("\n"); } }
```

ADDRESS CALCULATION IN 2D ARRAY

ROW MAJOR ORDER: In row-major storage, all elements of the first row are stored contiguously, followed by the elements of the second row, and so on. This layout privileges row-wise traversal and aligns well with cache-friendly access patterns in C-like languages.

Address of $A[I][J] = B + W * ((I - LR) * N + (J - LC))$

1. Row Major Order: Given an array, $arr[1.....10][1.....15]$ with base value 100 and the size of each element is 1 Byte in memory. Find the address of $arr[8][6]$ with the help of row-major order.

Given: Base address $B = 100$

- Storage size of one element store in any array $W = 1$ Bytes
- Row Subset of an element whose address to be found $I = 8$
- Column Subset of an element whose address to be found $J = 6$
- Lower Limit of row/start row index of matrix $LR = 1$
- Lower Limit of column/start column index of matrix $= 1$
- Number of column given in the matrix $N = (\text{Upper Bound} - \text{Lower Bound} + 1) = 15$

Formula: **ROW MASJOR ORDER**

$$\text{Address of } A[I][J] = B + W * ((I - LR) * N + (J - LC))$$

$$\begin{aligned}\text{Solution: Address of } A[8][6] &= 100 + 1 * ((8 - 1) * 15 + (6 - 1)) \\ &= 100 + 1 * ((7) * 15 + (5)) \\ &= 100 + 1 * (110)\end{aligned}$$

$$\text{Address of } A[I][J] = 210$$

Q. Compute the address of the element Arr[18][5] in a two-dimensional array Arr[20][5] stored in row-major order, we need to consider the base address, size of each element, number of rows, and number of columns.

Given:

Base address = 1000

Number of rows = 20

Size of each element = 2

Number of columns = 5

$$= 1000 + 180 + 10$$

$$= 1190$$

Therefore, the address of the element Arr[18][5] is 1190.

2. Column Major Order:

If elements of an array are stored in a column-major fashion means moving across the column and then to the next column then it's in column-major order. To find the address of the element using column-major order use the following formula:

$$\text{Address of } A[I][J] = B + W * ((J - LC) * M + (I - LR))$$

I = Row Subset of an element whose address to be found,

J = Column Subset of an element whose address to be found,

B = Base address, W = Storage size of one element store in any array(in byte),

LR = Lower Limit of row/start row index of matrix(If not given assume it as zero),

LC = Lower Limit of column/start column index of matrix(If not given assume it as zero),

M = Number of rows given in the matrix

Example: Given an array arr[1.....10][1.....15] with a base value of 100 and the size of each element is 1 Byte in memory find the address of arr[8][6] with the help of column-major order.

$$\begin{aligned}\text{Address of } A[8][6] &= 100 + 1 * ((6 - 1) * 10 + (8 - 1)) \\ &= 100 + 1 * ((5) * 10 + (7)) \\ &= 100 + 1 * (57)\end{aligned}$$

$$\text{Address of } A[I][J] = 157$$

Q1. A 2D array A is declared with row index starting from 1 and column index starting from 1.

The base address $B=1000$, element size $W=4$ bytes, and number of columns $N=6$. Calculate the address of $A[3][4]$.

Q2. Consider an array $A[0...4][0...7]$ stored in row-major order. If $B=2000$, $W=2$ bytes, $LR=0$, find the address of $A[2][5]$.

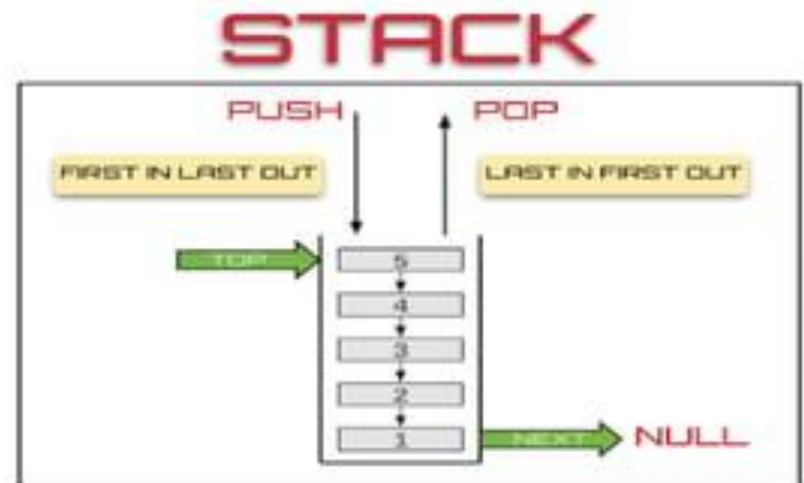
Q3. A matrix $M[1...5][1...10]$ is stored in row-major order. Given $B=500$ and $W=8$ bytes, compute the address of $M[4][7]$.

Q4. An array $M[10][15]$ is stored in row-major order. Derive the address calculation formula for $M[i][j]$ and compute the address when $i = 4$, $j = 7$, $BA = 1000$, and $w = 8$.

Q5. A matrix is stored in column-major order with 12 rows and 8 columns. If the address of $A[0][0]$ is 3000, determine the address of $A[6][3]$, assuming each element occupies 4 bytes.

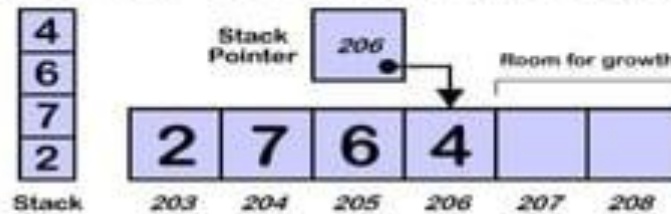
Stack

- Stack is a linear data structure which follows a particular order in which the operations are performed.
- Insertion of element into stack is called PUSH and deletion of element from stack is called POP.
- The order may be LIFO (Last In First Out) or FILO (First In Last Out).

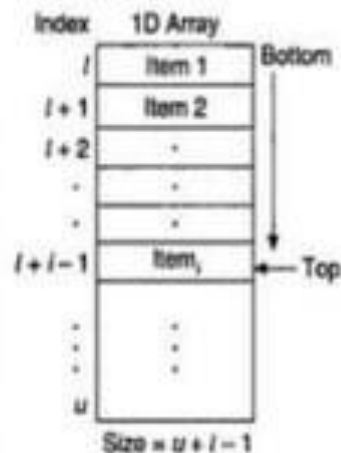


Representation of Stack in Memory

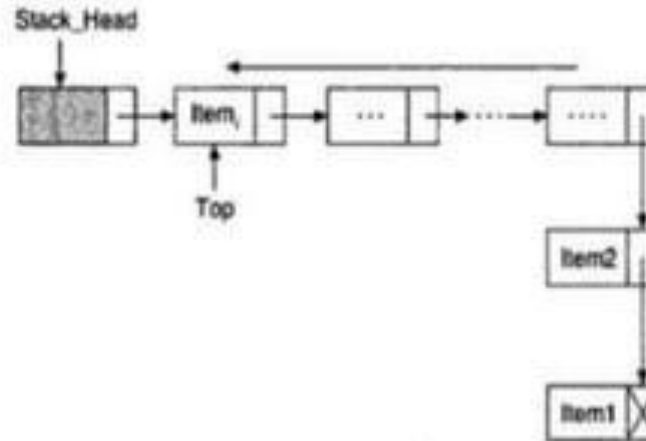
- The stack can be implemented into two ways:



- Using arrays (Static implementation)
- Using pointer (Dynamic implementation)



(a) Array representation of a stack



(b) Linked list representation of a stack



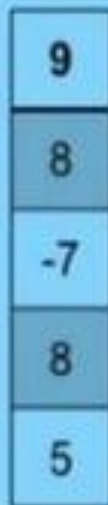
Operation on Stacks:

- `Stack( )`: It creates a new stack that is empty. It needs no parameter and returns an empty stack.
- `push(item)`: It adds a new item to the top of the stack.
- `pop( )`: It removes the top item from the stack.
- `peek( )`: It returns the top item from the stack but does not remove it.
- `isEmpty( )`: It tests whether the stack is empty.
- `size( )`: It returns the number of items on the stack.

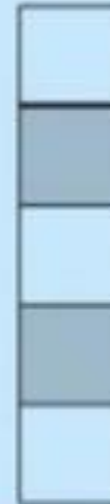
Stack Conditions

- Depending on implementation, may be necessary to check if stack is full -- attempt to add item to a full stack is an overflow error

Top
=>



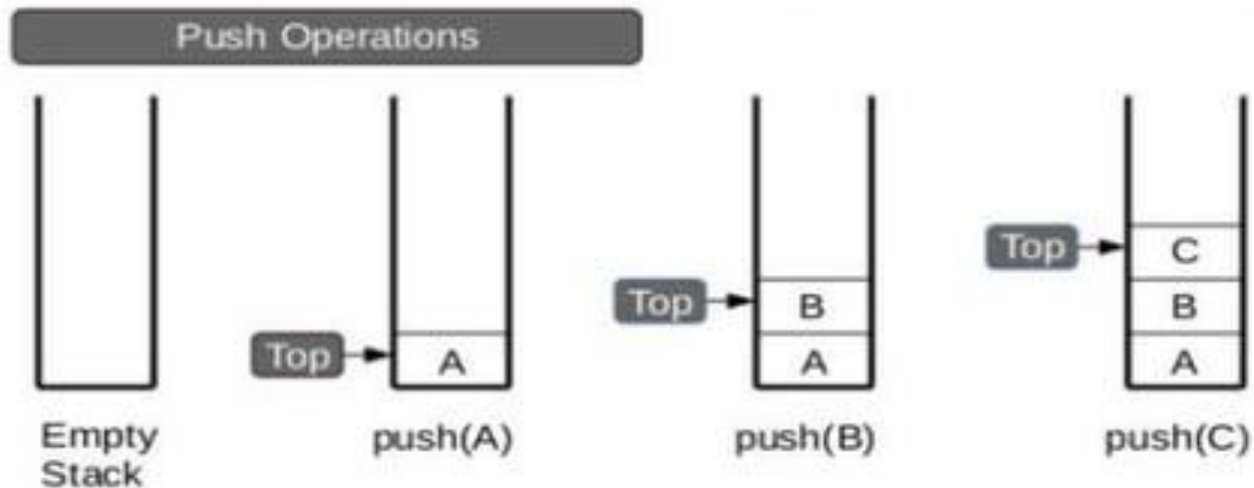
- Important to know if stack is empty -- attempt to remove an item from an empty stack is an underflow error.



Top =
NULL

PUSH Operation

- The process of adding one element or item to the stack is represented by an operation called as the PUSH operation.*



OVERFLOW STATE

If the stack is full and does not contain enough space to accept the given item

PUSH Operation:

- *The process of adding one element or item to the stack is represented by an operation called as the PUSH operation.*
- The new element is added at the topmost position of the stack.

ALGORITHM:

PUSH (STACK, TOP, SIZE, ITEM)

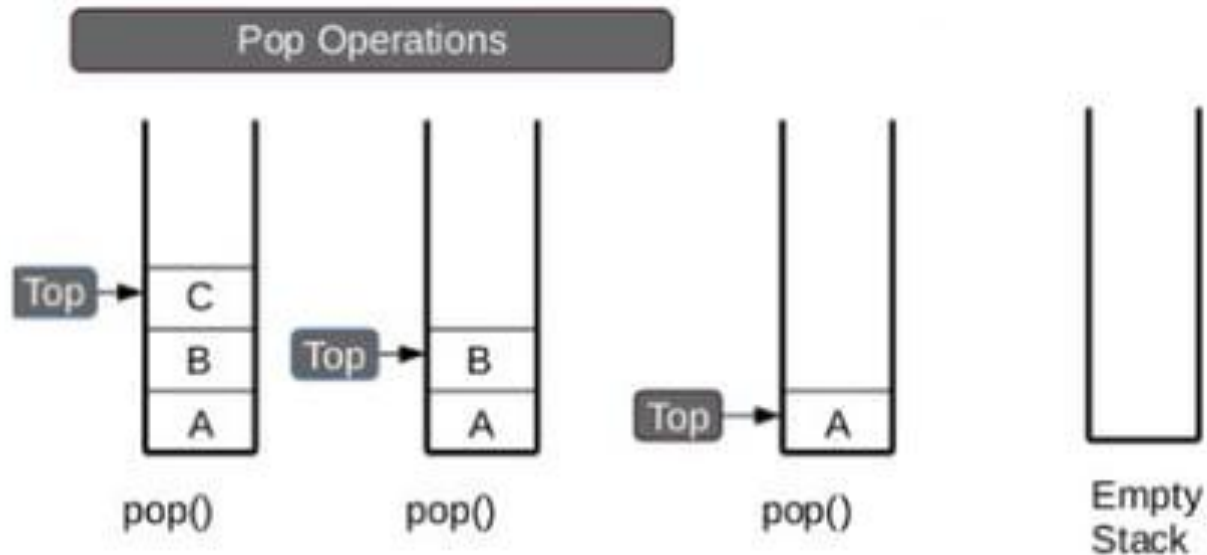
STACK is the array with N elements. TOP is the pointer to the top of the element of the array. ITEM to be inserted.

```
Step 1: if TOP = N then           [Check Overflow]
        PRINT " STACK is Full or Overflow"
        Exit
    [End if]
Step 2: TOP = TOP + 1             [Increment the TOP]
Step 3: STACK[TOP] = ITEM        [Insert the ITEM]
Step 4: Return
```

POP Operation

The process of deleting one element or item from the stack is represented by an operation called as the POP operation.

When elements are removed continuously from a stack, it shrinks at same end i.e., *top*



POP Operation

The process of deleting one element or item from the stack is represented by an operation called as the POP operation.

ALGORITHM: POP (STACK, TOP, ITEM)

STACK is the array with N elements. TOP is the pointer to the top of the element of the array. ITEM to be inserted.

Step 1: if $TOP = 0$ then [Check Underflow]

PRINT " STACK is Empty or Underflow"

Exit

[End if]

Step 2: $ITEM = STACK[TOP]$ [copy the TOP Element]

Step 3: $TOP = TOP - 1$ [Decrement the TOP]

Step 4: Return

PEEK Operation

The process of returning the top item from the stack but does not remove it called as the POP operation.

ALGORITHM: PEEK (STACK, TOP)

STACK is the array with N elements. TOP is the pointer to the top of the element of the array.

Step 1: if TOP = NULL then [Check Underflow]

PRINT “ STACK is Empty or Underflow”

Exit

[End if]

Step 2: Return (STACK[TOP]) [Return the top element of the stack]

Step 3:Exit

Application of Stacks

- It is used to reverse a word. You push a given word to stack – letter by letter and then pop letter from the stack.
- “Undo” mechanism in text editor.
- Backtracking: This is a process when you need to access the most recent data element in a series of elements. Once you reach a dead end, you must backtrack.
- Language Processing: Compiler’ syntax check for matching braces is implemented by using stack.
- Conversion of decimal number to binary.
- To solve tower of Hanoi.
- Conversion of infix expression into prefix and postfix.
- Quick sort
- Runtime memory management.

Arithmetic Expression

- An expression is a combination of operands and operators that after evaluation results in a single value.
- Operand consists of constants and variables.
- Operators consists of {, +, -, *, /,),] etc.
- Expression can be

Infix Expression: If an operator is in between two operands, it is called infix expression.

- Example: $a + b$, where a and b are operands and $+$ is an operator.

Postfix Expression: If an operator follows the two operands, it is called postfix expression.

- Example: $ab +$

Prefix Expression: an operator precedes the two operands, it is called prefix expression.

- Example: $+ab$

Algorithm to evaluate postfix expression

1. Read a character

2. If the character is a digit, convert the character into int and push the integer into the stack.

3. If the character is an operator,

- Pop the elements from the stack twice obtaining two operands.
- Perform the operation
- Push the result into the stack.

Let's understand the evaluation of postfix expression using stack.

Example 1:

Postfix

expression:

3 4 * 2 5 * +

The result of the above expression is 22.

Input	Stack	
3 4 * 2 5 * +	empty	Push 3
4 * 2 5 * +	3	Push 4
* 2 5 * +	4 3	Pop 3 and 4 from the stack and perform $3 * 4 = 12$. Push 12 into the stack.
2 5 * +	12	Push 2
5 * +	2 12	Push 5
* +	5 2 12	Pop 5 and 2 from the stack and perform $5 * 2 = 10$. Push 10 into the stack.
+	10 12	Pop 10 and 12 from the stack and perform $10 + 12 = 22$. Push 22 into the stack.

Example 1: Postfix expression: 2 3 4 * +

The result of the above expression is 14.

Input	Stack	
2 3 4 * +	empty	Push 2
3 4 * +	2	Push 3
4 * +	3 2	Push 4
* +	4 3 2	Pop 4 and 3, and perform $4*3 = 12$. Push 12 into the stack.
+	12 2	Pop 12 and 2 from the stack, and perform $12+2 = 14$. Push 14 into the stack.

Rules for the conversion from infix to postfix expression

1. Print the operand as they arrive.
2. If the stack is empty or contains a left parenthesis on top, push the incoming operator on to the stack.
3. If the incoming symbol is '(', push it on to the stack.
4. If the incoming symbol is ')', pop the stack and print the operators until the left parenthesis is found.
5. If the incoming symbol has higher precedence than the top of the stack, push it on the stack.
6. If the incoming symbol has lower precedence than the top of the stack, pop and print the top of the stack. Then test the incoming operator against the new top of the stack.
7. If the incoming operator has the same precedence with the top of the stack then use the associativity rules. If the associativity is from left to right then pop and print the top of the stack then push the incoming operator. If the associativity is from right to left then push the incoming operator.
8. At the end of the expression, pop and print all the operators of the stack.

EXAMPLE: Let us illustrate the procedure *InfixToPostfix* with the following arithmetic expression:

Input: $(A + B)^C - (D * E) / F$ (infix form)

Read symbol	Stack	Output
Initial	(	
1	((	
2	((	A
3	((+	A
4	((+	AB
5	(	AB+
6	(^	AB+
7	(^	AB + C
8	(-	AB + C ^
9	(- (	AB + C ^
10	(- (	AB + C ^ D
11	(- (*	AB + C ^ D
12	(- (*	AB + C ^ DE
13	(-	AB + C ^ DE *
14	(- /	AB + C ^ DE *
15	(- /	AB + C ^ DE * F
16		AB + C ^ DE * F / -

Infix expression: K + L - M*N + (O^P) * W/U/V * T + Q		
Input Expression	Stack	Postfix Expression
K		K
+	+	
L	+	K L
-	-	K L +
M	-	K L + M
*	- *	K L + M
N	- *	K L + M N
+	+	K L + M N * -
(	+ (	K L + M N * -
O	+ (	K L + M N * - O
^	+ (^	K L + M N * - O
P	+ (^	K L + M N * - O P
)	+	K L + M N * - O P ^
*	+ *	K L + M N * - O P ^
W	+ *	K L + M N * - O P ^ W
/	+ /	K L + M N * - O P ^ W *
U	+ /	K L + M N * - O P ^ W * U
/	+ /	K L + M N * - O P ^ W * U /
V	+ /	K L + M N * - O P ^ W * U / V
*	+ *	K L + M N * - O P ^ W * U / V /
T	+ *	K L + M N * - O P ^ W * U / V / T
+	+	K L + M N * - O P ^ W * U / V / T * K L + M N * - O P ^ W * U / V / T * +
Q	+	K L + M N * - O P ^ W * U / V / T * Q
		K L + M N * - O P ^ W * U / V / T * + Q +

Convert the infix expression $(A + B / C * (D + E) - F)$ to postfix expression using stack.

Input	Stack	Postfix expression
		.

Evaluate the postfix expression and show the contents of the stack for the following postfix expression $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ * 2\ ^\ 3$.

Introduction to Recursion

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function. Using a recursive algorithm, certain problems can be solved quite easily.

Examples of such problems are Towers of Hanoi (TOH), [Inorder/Preorder/Postorder Tree Traversals](#), DFS of Graph, etc

Recursion is a technique with the help of which we can reduce the length of our code and make it easier to read and write. It has certain advantages over the iteration technique (Looping).

EX- Recursive Function for finding Factorial

```
int fact(int n)
{
    if (n < = 1)  // base case
        return 1;
    Else
        return n*fact(n-1);
}
```

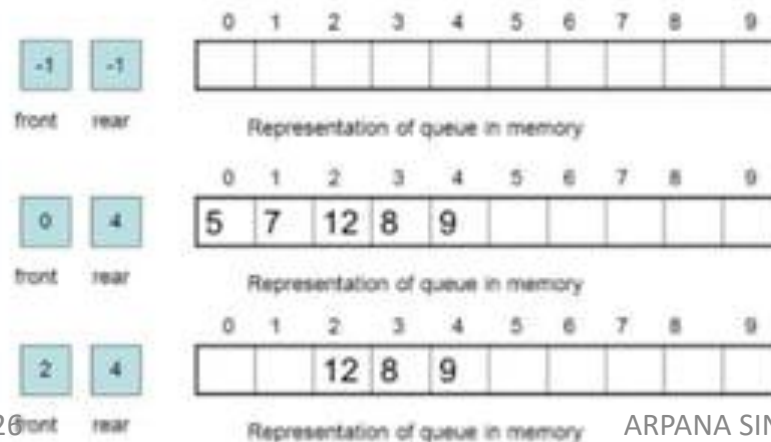
Queue

- *A queue is an ordered collection of items where an item is inserted at one end called the “rear” and an existing item is removed at the other end, called the “front”.*
- Queue is also called as FIFO list i.e. First-In First-Out.
- In the queue only two operations are allowed enqueue and dequeue.
- Enqueue means to insert an item into back of the queue.
- Dequeue means removing the front item. The people standing in a railway reservation row are an example of queue.

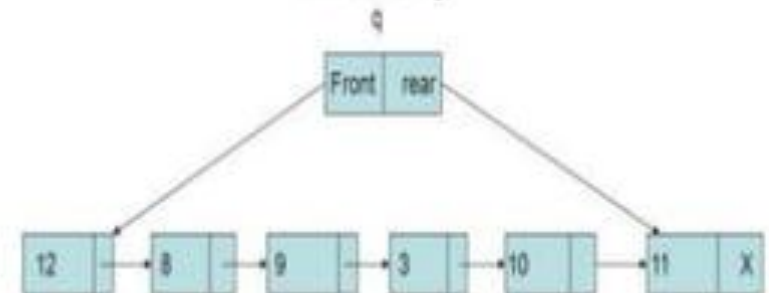
Queue

- The queue can be implemented into two ways:
 - Using arrays (Static implementation)
 - Using pointer (Dynamic implementation)

Array representation of linear queue



Representation of a queue in memory



Types of Queues

- Queue can be of four types:
 - Simple Queue
 - Circular Queue
 - Priority Queue
 - De-queue (Double Ended Queue)